

Corrigé — Contrôle 2nde : Boucle For

Exercice 1 — Fonctions somme

1. Analyse des trois fonctions :

- **Fonction a.** — $\text{range}(6) = [0, 1, 2, 3, 4, 5] \rightarrow S = 0 + 1 + 2 + 3 + 4 + 5 = 15 \checkmark$ — donne 15
 - **Fonction b.** — $\text{range}(1, 8, 2) = [1, 3, 5, 7] \rightarrow S = 1 + 3 + 5 + 7 = 16 \times$ — ne donne pas 15
 - **Fonction c.** — $\text{range}(1, 6) = [1, 2, 3, 4, 5] \rightarrow S = 1 + 2 + 3 + 4 + 5 = 15 \checkmark$ — donne 15
- Les fonctions qui affichent 15 sont **a** et **c**.

2. La fonction **b** affiche 16.

- $\text{range}(1, 8, 2)$ génère les entiers impairs de 1 à 7 inclus : 1, 3, 5, 7. Leur somme = 16.

Exercice 2 — Fonction nbre_e

2a. Résultat de `nbre_e("Je n'aime pas les maths")` :

- La fonction parcourt chaque caractère de la chaîne et compte les "e" minuscules. Dans "Je n'aime pas les maths" : le « e » de **Je**, celui de **aime** et celui de **les** → **3**. La fonction renvoie **3**.

2b. Explication :

- La boucle `for lettre in phrase` itère sur chaque caractère. `if lettre=="e"` ne teste que le « e » minuscule — le « E » majuscule ne serait pas compté. Le compteur `nbre_e` s'incrémente à chaque « e » trouvé.

3. Avec « Je ne vais pas à la plage » :

- Les « e » minuscules présents : **Je**, **ne**, **plage** → **3** occurrences. La fonction renvoie 3.
- *Attention* : le « à » (avec accent) n'est pas un « e » ; plus généralement, un caractère accentué comme « é » **ne serait pas compté** car `"é" != "e"` — ce sont deux caractères Unicode distincts.

Exercice 3 — Fonction som

1. Tableau de trace pour $n = 6$:

k	init	1	2	3	4	5	6
s	0	1	3	6	10	15	21

- À chaque itération k , s accumule les valeurs : $s = 0 + 1 = 1$, $s = 1 + 2 = 3$, $s = 3 + 3 = 6$, $s = 6 + 4 = 10$, $s = 10 + 5 = 15$, $s = 15 + 6 = 21$.

2. Cet algorithme calcule la somme des n premiers entiers : $1 + 2 + 3 + \dots + n$.

► Pour $n = 6$: $1 + 2 + 3 + 4 + 5 + 6 = 21$. Formule : $\frac{n(n+1)}{2} = \frac{6 \times 7}{2} = 21$. ✓

3a. Algorithme modifié pour la somme des n premiers entiers impairs :

```

1 def som_impairs(n):
2     s = 0
3     for k in range(n):           # k parcourt 0, 1, 2, ..., n-1
4         s = s + 2*k + 1          # 2k+1 : definition d'un entier impair
5     return s

```

► Tout entier impair s'écrit $2k+1$ avec k entier. Pour $k \in \{0, 1, \dots, n-1\}$ on obtient les n premiers entiers impairs : $1, 3, 5, \dots, 2n-1$.

3b. Pour $n = 7$:

► Impairs : $1 + 3 + 5 + 7 + 9 + 11 + 13 = 49$. Formule : $n^2 = 7^2 = 49$. ✓

Exercice 4 — Multiples de 3 non multiples de 9

1. La ligne 5 teste : $k\%3==0$ and $k\%9!=0$

- $k\%3==0$: k est un multiple de 3.
- $k\%9!=0$: k n'est pas un multiple de 9.
- La condition sélectionne les entiers de 1 à 50 qui sont **multiples de 3 mais pas de 9**.

2. Le programme affiche tous ces entiers puis leur nombre total :

- Multiples de 3 dans $[1, 50]$: 3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48 (16 valeurs)
- À exclure (multiples de 9) : 9, 18, 27, 36, 45 (5 valeurs)
- Valeurs affichées : **3, 6, 12, 15, 21, 24, 30, 33, 39, 42, 48** → 11 valeurs. La fonction retourne **11**.

Exercice 5 — Budget loisirs d'Oscar

1. Budget la 2^e année (après une réduction de 5 %) :

► $B_2 = 1800 \times (1 - 0,05) = 1800 \times 0,95 = 1710$ €

2. Budget total sur les 2 premières années :

- $T_2 = 1800 + 1710 = 3510$ €
- Année 1 : 1800 €. Année 2 : $1800 \times 0,95 = 1710$ €. Total = 3510 €.

3. Programme complété :

```

1 def budget():
2     B = 1800
3     T = 0
4     for i in range(1, 11):          # 10 années
5         T = T + B
6         B = B * 0.95                # réduction de 5% chaque année
7     return T

```

- À chaque tour de boucle : on ajoute le budget de l'année courante à T , puis on réduit B de 5 % pour l'année suivante. `range(1, 11)` génère 10 itérations (années 1 à 10).
- Calcul : $T = 1800 + 1710 + 1624,5 + 1543,3 + \dots$ sur 10 ans.
- Formule : $T = 1800 \times \frac{1 - 0,95^{10}}{1 - 0,95} \approx 1800 \times 7,95 \approx \mathbf{14\ 306\ €}$ (environ).

Exercice 6 — Pyramide d'oranges1. Nombre d'oranges à l'étage i :

- L'étage i est un carré de i oranges de côté : il contient donc i^2 oranges (étage 1 : $1^2 = 1$; étage 2 : $2^2 = 4$; étage 3 : $3^2 = 9$, etc.).

2. Fonction `pyramide` complétée :

```

1 def pyramide(n):
2     nbre_oranges = 0
3     for i in range(1, n+1):        # etages 1 a n
4         nbre_oranges = nbre_oranges + i**2
5     return nbre_oranges

```

- `range(1, n+1)` parcourt les étages de 1 à n . À chaque tour, on ajoute i^2 (le nombre d'oranges de l'étage i) à l'accumulateur `nbre_oranges`.

3. Nombre d'oranges pour une pyramide de dix étages :

- $\text{pyramide}(10) = 1^2 + 2^2 + 3^2 + \dots + 10^2 = 1 + 4 + 9 + 16 + 25 + 36 + 49 + 64 + 81 + 100 = \mathbf{385}$
- Formule : $\frac{n(n+1)(2n+1)}{6} = \frac{10 \times 11 \times 21}{6} = 385. \checkmark$